SAE S1.01/02

Implémentation d’un besoin client
Comparaison d’approches algorithmiques

Janvier 2024

Drouiche Ilyès
Fernandes Mathias
Groupe A1

Table of contents

Introduction.....2
1. Things we did.....3
i) Monday.....3
ii) Tuesday 3
iii) Wednesday.....3
iv) Thursday.....4
2. Results analysis.....4
i) Manual glossary.....4
ii) Automatic glossary without sort.....5
iii) Automatic glossary with sort.....5
iv) Execution time.....5
3. Comparison number.....6
4. Development prospects.....6

1. Introduction

In this SAE, we have to make a program which does an automatic classification of dispatch.

To achieve this classification, the program uses words to indicate the membership of a particular category.
Each word is assigned to a weight corresponding to the degree of word which indicates the membership of the category.
Three weights are possible: 1, 2 or 3. A maximum weight of 3 indicates the word is very strongly associated with the category.
For example, the glossary for the SPORTS category could be made up of the following words and associated weights: sport:3, cup:3, match:3, player:2, home:1.

To do that, we have 2 parts.

The first part consists of making a glossary manually. So to make that we start by creating a glossary for each category with all the words which correspond to it for us. After that, we test it with “depeches.txt”, using the category written on every dispatch to compare, to finally find 60% of good answers.

The second part consists of making a glossary automatically. For each category, we range “depeches.txt” where the category of the dispatch is the same as the actual category and add all its words to a list in order to create the glossary. After that for each glossary we count the number of occurrences of each word and update the value positively if the dispatch is from the good category or decrease it if the dispatch is from another category. We finally create all glossaries by incrementing the score of each word base of the score at the prior step.

2. Things we did

i) Monday

Mathias&me made the first part except the last method. We didn’t run into a lot of issues so it was just understanding the subject and starting to work on it.

ii) Tuesday

-We made the last method of the first part and also the first three functions of the second part. Moreover we encounter our first major problem with the last method of the first part because of the comparison for the category “sport”. Finally we discovered that the problem was due to a writing error for the dictionary which was written without a s at the end(sports).
Mathias also makes a function called listEntierPourChaqueCat which creates a table which contains a PaireChaineEntier for each category on the list categories in parameters.

```
public static ArrayList<PaireChaineEntier> listEntierPourChaqueCat(ArrayList<Categorie> categories) {
    ArrayList<PaireChaineEntier> listEntier = new ArrayList<>();
    for (int i = 0; i < categories.size(); i++) {
        listEntier.add(i, new PaireChaineEntier(categories.get(i).getNom(), entier: 0));
        //System.out.println(listEntier.get(0).getChaine());
    }
    return listEntier;

}
```

Ilyes started this document by making the introduction .

iii) Wednesday

We encountered another error. This time it was due to all words on the dispatch which contained the char ‘:’ because it was our way to separate the chain to the integer for the glossaries and it caused it to give the value ‘:3’ to the integer, which caused an error.
So we replaced the value of the separator to last

```
public void initLexique(String nomFichier) {
    //creation d'un tableau de dépêches
    //Categorie<PaireChaineEntier> depeches = new ArrayList<>();
    try {
        // lecture du fichier d'entrée
        FileInputStream file = new FileInputStream(nomFichier);
        Scanner scanner = new Scanner(file);
        this.lexique = new ArrayList<>();
        while (scanner.hasNextLine()) {
            String ligne = scanner.nextLine();
            //System.out.println(ligne);
            //Partie 2 erreur avec :: donc on va check pour prendre le dernier : de la chaine
            int separateur = ligne.lastIndexOf( ch: ':');            //Pour la question

            int poids = Integer.parseInt(ligne.substring( beginIndex: separateur+1));
            String mot = ligne.substring(0,separateur);
            PaireChaineEntier unePaireChaineEntier = new PaireChaineEntier(mot,poids);
            this.lexique.add(unePaireChaineEntier);
        }
        scanner.close();
        //UtilitairePaireChaineEntier.affichageListesPaire(lexique);
    } catch (IOException e) {
        e.printStackTrace();
    }
```

iv) Thursday

I(Ilyes) starts a bubble sort and an algorithm to make a binary search. We didn't use the bubble sort finally because it just increases the execution time so instead we choose to place all the elements at their place order when we added them.

```
public static void triElement(ArrayList<PaireChaineEntier> lexiqueTri, String element) {
    // { lexique triée} => { élément inséré au bon indice si il n'existe pas déjà
    // en utilisant la méthode DU TRI À BULLES AMÉLIORÉ
    boolean onContinue = true;
    int compa;
    int j = 0;
    while (onContinue && j<lexiqueTri.size()) { //On stop sans rajouter
                                                 // l'élément si lexiqueTri.get(j).getChaine()=élément
        onContinue = false;
        compa = element.compareTo(lexiqueTri.get(j).getChaine());
        if ( compa > 0) {               //Quand element>lexiqueTri.get(j).getChaine() on continue
            onContinue   = true;
        } else if(compa<0){
            lexiqueTri.add(j, new PaireChaineEntier(element,  entier: 0));
        }
        j++;
    }
}
```

3. Results analysis

i) Manual glossary

With the manual glossary, we have a 60.4% average of right dispatch in a specific category. This is the worst average compared to the glossary generated automatically.

```
CULTURE:        49%
ENVIRONNEMENT-SCIENCES:        31%
POLITIQUE:        54%
SPORTS:      80%
ECONOMIE:        88%
MOYENNE:         60.4%
```

ii) Automatic glossary without sort

The glossary generated automatically without sort gives us a 98% average of right dispatch in a specific category.

This is the best average of right dispatch but it takes 856 ms to be executed.

```
CULTURE:        98%
ENVIRONNEMENT-SCIENCES:        97%
POLITIQUE:        99%
SPORTS:        100%
ECONOMIE:        96%
MOYENNE:        98.0%
```

iii) Automatic glossary with sort

The glossary generated automatically with sort gives us a 78.4% average of right dispatch in a specific category.

```
CULTURE:        88%
ENVIRONNEMENT-SCIENCES:        14%
POLITIQUE:        94%
SPORTS:        96%
ECONOMIE:        100%
MOYENNE:        78.4%
```

iv) Execution time

This average is lesser than the glossary generated without sorting but it takes a shorter time to be executed (only 750 ms).

```
DEBUT lexique Non Trie
votre saisie a été réalisée en : 856ms
DEBUT lexique Trie
votre saisie a été réalisée en : 750ms
```

4. Comparison number

We study the number of comparisons for the method CalculScores without sorting the list for each category.

```
chargement des dépêches
Il y a eu 11083583 comparaisons.
Il y a eu 12562781 comparaisons.
Il y a eu 11729410 comparaisons.
Il y a eu 11274417 comparaisons.
Il y a eu 11655292 comparaisons.
```

```java
public static void calculScoresCompteur (ArrayList < Depeche > depeches, String categorie, ArrayList < PaireChaineEntier > dictionnaire){
    //{} => {met à jour les scores des mots présents dans dictionnaire. Lorsqu'un mot présent dans dictionnaire apparaît dans une dépêche de depeches
    //son score est : décrémenté si la dépêche n'est pas dans la catégorie categorie
    //incrémenté si la dépêche est dans la catégorie categorie.
    int nbComparaisons = 0;
    PaireResultatCompteur<Integer> compteurIndMot = new PaireResultatCompteur<>(res: 0, compteur: 0);
    for (int i = 0; i < depeches.size(); i++) {
        nbComparaisons++;
        for (int r = 0; r < depeches.get(i).getMots().size(); r++) {
            nbComparaisons++;
            compteurIndMot = UtilitairePaireChaineEntier.indicePourChaineNbComp(dictionnaire, depeches.get(i).getMots().get(r));
            nbComparaisons += compteurIndMot.getCompteur();     //On rajoute le nombre de compa d'indicePourChaineNbComp à nbCompa
            if (compteurIndMot.getRes() != -1) {          //Si le mot n'est pas dans le dictionnaire alors on passe juste au suivant
                if (depeches.get(i).getCategorie().compareToIgnoreCase(categorie) == 0) {   //Si la depeche dans laquelle est située le mot a la catégorie
                    dictionnaire.get(compteurIndMot.getRes()).ajoutEntier(nb: 1);                          //qu'on observe alors on rajoute 1
                } else {                                                                  //Sinon on enlève 1
                    dictionnaire.get(compteurIndMot.getRes()).ajoutEntier(nb: -1);       //Pas de comparatisons dans ajoutEntier
                }
                nbComparaisons++;
            }
            nbComparaisons++;
        }
    }
    System.out.println("Il y a eu " + nbComparaisons + " comparaisons.");
}
```

```java
public static PaireResultatCompteur<Integer> indicePourChaineNbComp(ArrayList<PaireChaineEntier> listePaires, String chaine) {
    //{} => {retourne l'indice de chaine dans listePaires si chaine est présente et -1 sinon.}

    int i = 0;
    int nbComparaison=0;
    while (i < listePaires.size() && listePaires.get(i).getChaine().compareToIgnoreCase(chaine) != 0) {
        nbComparaison++;            //On compte qu'une seule compa à chaque boucle while
        i++;
    }
    nbComparaison++;        //Compa lors de l'arrêt de la boucle
    nbComparaison++;        //Concerne la condition juste en dessous
    if (i == listePaires.size()) {
        return new PaireResultatCompteur<>(res: -1,nbComparaison);
    } else {
        return new PaireResultatCompteur<>(i,nbComparaison);
    }
}
```

5. Development prospects

To upgrade our system, we could start by correcting the dispatches on the sort method which didn't register all the words in the category and give us only 80% of good answers..

After that, we could add a way to return an ArrayList<String> instead of a String when there is more than one category with the function chaineMax to adjust the number of good answers we would find.